

CSC343Y5 Final Exam Review Session

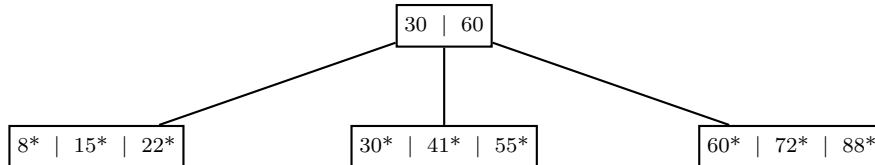
University of Toronto Mississauga

Module D: Indexing and Physical Design

Question D1.

[6 marks]

Consider the following B+ tree.



(a) [2 marks] Insert the key 47. Draw the resulting tree.

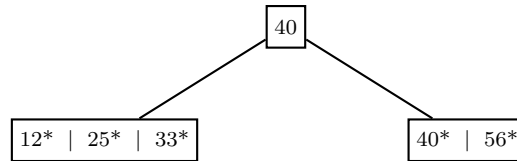
(b) [3 marks] Now insert 12 into the tree from part (a). Draw the resulting tree.

(c) [1 mark] Name the key that was copied up and the key that was pushed up, and state the reason the two cases are treated differently.

Question D2.

[4 marks]

Consider this separate, smaller B+ tree (same conventions).



- (a) **[3 marks]** Delete 56. Draw the resulting tree and state which repair you used and why the other one was not needed.

- (b) **[1 mark]** Suppose instead the left leaf had been `[12*|25*]`. What would the deletion of 56 have produced?

Question D3.

[5 marks]

An online quiz platform stores `Attempt(attempt_id, student_email, quiz_id, submitted_at, score)` with roughly 50 million rows. Four queries dominate:

- **Q1:** fetch the one attempt with a given `attempt_id` (millions of times per day).
- **Q2:** fetch all attempts of one `student_email` (about 30 rows each time).
- **Q3:** fetch every attempt with `submitted_at` in the last 24 hours (about 0.5% of rows), for a nightly report.
- **Q4:** fetch every attempt with `score > 0` (about 97% of rows), for an annual archive job.

- (a) **[3 marks]** For Q1, Q2 and Q3, state which index structure you would build (hash or B+ tree) and justify each in one sentence.

(b) [**1 mark**] For Q4, what would you build, and why?

(c) [**1 mark**] You may cluster the table on exactly one column. Would clustering on `submitted_at` or on `student_email` do more good?

Module E: Functional Dependencies and Normalization

Question E1.

[8 marks]

Consider $R(A, B, C, D, E)$ with

$$F = \{ A \rightarrow B, \quad B \rightarrow C, \quad CD \rightarrow E, \quad E \rightarrow D \}.$$

- (a) [2 marks] Compute $\{A\}^+$ and $\{A, D\}^+$, showing the dependency used at each step.
- (b) [2 marks] State every candidate key of R , and list the prime and non-prime attributes.
- (c) [2 marks] What is the highest normal form R satisfies? Name the dependency that blocks the next form and state precisely which condition fails.
- (d) [2 marks] Decompose R into BCNF, showing each step. State which dependency of F your final

schema fails to preserve.

Question E2.

[5 marks]

Consider $R(A, B, C, D)$ with

$$G = \{ A \rightarrow B, \quad A \rightarrow C, \quad B \rightarrow C, \quad BC \rightarrow D, \quad D \rightarrow A \}.$$

Every right-hand side is already a single attribute, so you may skip that step.

- (a) [1.5 marks] One dependency is redundant. Identify it and justify with a closure computed without it.

- (b) [1.5 marks] One dependency has an extraneous attribute on its left-hand side. Identify it, say which attribute goes, and justify with a closure. Show also that the *other* attribute cannot be dropped instead.

- (c) [1 mark] State the resulting minimal basis.

- (d) [**1 mark**] Using your minimal basis, give every candidate key of R .

Question E3.

[3 marks]

$R(A, B, C)$ satisfies only $A \rightarrow B$. Someone decomposes it into $R_1(A, B)$ and $R_2(B, C)$.

- (a) [**1.5 marks**] Apply the shared-attributes test and state whether the decomposition is lossless.
- (b) [**1.5 marks**] Give a two-row instance of R whose projections rejoin to something larger, and name a spurious tuple.

Module F: Transactions and Concurrency Control

Question F1.

[5 marks]

Consider

$$S = R1(X) \quad W2(X) \quad R3(Y) \quad W1(Y) \quad R2(Z) \quad W3(Z)$$

- (a) [3 marks] List every conflicting pair with its object, type and edge, then draw the precedence graph.

Object	Pair, in schedule order	Type	Edge
_____	_____	_____	_____
_____	_____	_____	_____
_____	_____	_____	_____

T_1

T_2

T_3

- (b) [1 mark] Is S conflict serializable? Justify in one sentence.

- (c) [1 mark] Name a single operation whose removal from S would make it conflict serializable, and give the resulting serial order.

Question F2.

[4.5 marks]

For each schedule, state whether it is recoverable, whether it avoids cascading aborts, and whether it is strict, and give the operation or commit that decides the case.

Schedule	Recov.	ACA	Strict	What decides it
(i) R1(A) W1(A) R2(A) W2(A) C1 C2	_____	_____	_____	
(ii) W1(A) W1(B) C1 R2(A) W2(B) C2	_____	_____	_____	
(iii) R1(A) W2(A) R2(B) W1(B) C2 C1	_____	_____	_____	

Question F3.

[6 marks]

Two transactions run with locking. S(A) is a shared lock, X(A) an exclusive lock, U(A) the release. Time runs downward.

T1	T2
S(A), R(A), U(A)	X(B), R(B), W(B), U(B)
X(B), R(B), W(B), U(B), Commit	X(A), R(A), W(A), U(A), Commit

- (a) [2 marks] Give the edge contributed by A and the edge contributed by B, name the conflict pattern behind each, and state whether the schedule is conflict serializable.

- (b) [2 marks] Neither transaction obeys Two-Phase Locking. For *each*, identify the first release and the later acquisition that breaks the rule, and state the rule.

- (c) [2 marks] Now suppose both transactions follow *Strict* 2PL, keeping the same operations in the

same per-transaction order. Trace what happens. Does a serial order result?

Question F4.

[4 marks]

- (a) [2 marks] Three transactions run under strict 2PL. T_1 holds a lock on **Course** and wants **Section**; T_2 holds **Section** and wants **Room**; T_3 holds **Room** and wants **Course**. Draw the waits-for graph, state what the DBMS concludes, and state what it does.
- (b) [2 marks] Give one change to the *application* that would prevent this from arising, and explain why it works. Then state why a lock timeout is not the same kind of answer.

End of review handout.